DevIA Radar

Como desenvolvedores discutem Inteligência Artificial no desenvolvimento de software

Etapa 2 · spaCy, embeddings e transformers · issues do GitHub

Augusto Blois · Pedro Flávio Nogueira Ribeiro de Luna
Dados Não Estruturados · UFPB · 2026

979

issues do GitHub, as mesmas da etapa 1

34.460

frases lidas pelo transformer

4

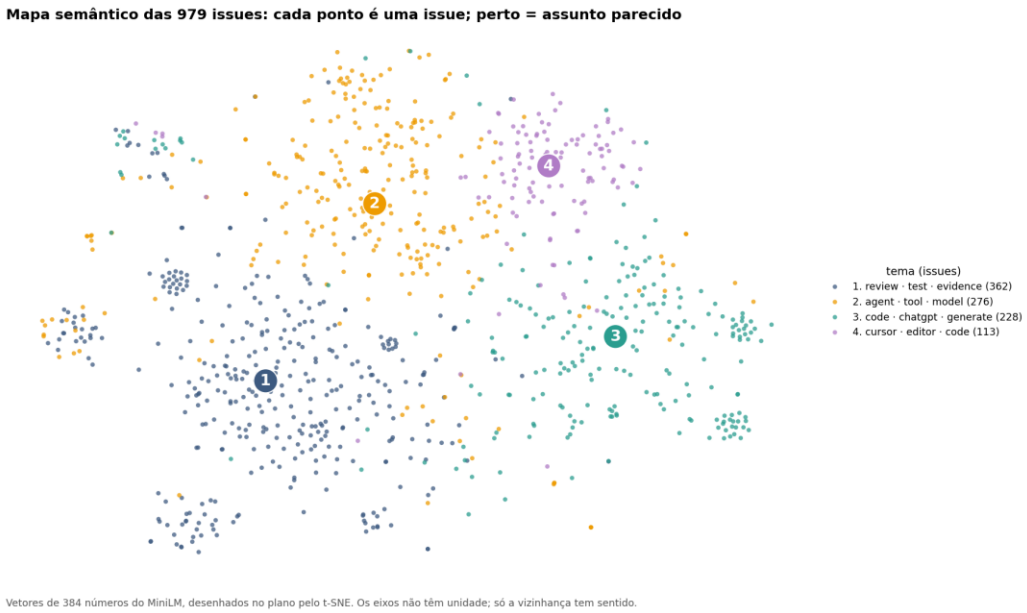
temas achados com os embeddings

O que esta etapa investiga

Quando o computador lê pelo sentido, do que as issues falam, e em que tom?

Por que as mesmas issues da etapa 1

- A etapa 1 somou o tom das palavras de cada issue: 65% saíram positivas.
- Com a mesma fonte, dá para comparar o léxico com modelos que leem a frase inteira.
- O texto já chega limpo: a etapa 1 apagou o código, os links e os logs.



As 979 issues no mapa do t-SNE, com a cor do tema do KMeans.

979
issues da etapa 1

39.279
frases do parser

34.460
com 6 palavras ou mais

Uma issue mistura pedido, elogio e queixa. A etapa 2 lê frase a frase, e a frase com menos de 6 palavras ("Thanks!") fica de fora.

Teoria e parâmetros

Três técnicas, aplicadas em sequência sobre as mesmas frases

1 Pipeline do spaCy

- Linha de montagem: cada componente anota o texto e passa adiante.
- O parser corta as frases. O ner marca entidades, como empresa e pessoa.
- entity_ruler: regra nossa, antes do ner, que marca as ferramentas de IA.
- vader: componente nosso que dá a nota de cada frase.

2 Embeddings

- Embedding: lista de números que representa o sentido de um texto.
- O cosseno mede se dois vetores apontam para o mesmo lado.
- Estático: um vetor por palavra. Contextual: um por frase inteira.
- KMeans: junta em temas as issues de vetor parecido.

3 Transformer

- Cada pedaço de palavra olha para todos os outros: a atenção.
- RoBERTa: leu 160 GB de texto e foi ajustado com tweets rotulados.
- Devolve uma nota por classe (o logit). A softmax vira as notas em probabilidades.
- As mesmas 3 classes do VADER: negativo, neutro e positivo.

Parâmetros do projeto

en_core_web_md · frase ≥ 6 palavras · MiniLM: 384 números · KMeans: 4 a 12 temas (silhueta) · RoBERTa: 3 classes · VADER: ±0,05

Implementação 1: o pipeline do spaCy

Arquivo pipeline.py: o modelo pronto, com dois componentes nossos

```
nlp = spacy.load("en_core_web_md")
regras = nlp.add_pipe("entity_ruler", before="ner",
          config={"phrase_matcher_attr": "LOWER"})
regras.add_patterns(padroes_ferramentas())
nlp.add_pipe("vader", last=True)
for frase in nlp(texto).sents:  # do parser
    print(frase.ents, frase._.vader)
```

O entity_ruler entra antes do ner, e o ner respeita o que a regra marcou como FERRAMENTA. O vader, componente nosso, guarda em frase._.vader a nota de cada frase, de -1 a +1.

Onde aprender

spacy.io/usage/processing-pipelines, a documentação do pipeline

```
opção 2 do menu, a frase de exemplo:
"Claude Code keeps rewriting my tests..."

sem a regra (só o ner do modelo)
  Claude Code      PERSON  pessoa
  GitHub Copilot   ORG     empresa

com o entity_ruler antes do ner
  Claude           FERRAMENTA
  GitHub Copilot   FERRAMENTA
```

O NER DO MODELO NAS MENÇÕES AO CLAUDE

44% PERSON (pessoa), sem a regra

0% com o entity_ruler antes do ner

Implementação 2: embeddings e transformer

Arquivos embeddings.py e sentimento.py: da frase ao vetor e à classe




```
# embeddings.py: 384 números por frase
vetores_frases = codificar(frases)    # MiniLM
notas = vetores_frases @ codificar([consulta])[0]
# sentimento.py: a frase pelo RoBERTa
entrada = tokenizador(frase, return_tensors="pt")
logits = modelo(**entrada).logits[0]    # 3 notas
probabilidades = torch.softmax(logits, dim=-1)
```

- O MiniLM vira cada frase em 384 números. O cosseno compara os vetores, e o KMeans junta as issues em 4 temas.
- O RoBERTa lê a frase em pedaços ("Cla" e "ude") e dá uma nota por classe. A softmax vira as notas em probabilidades.

Onde aprender: sbert.net e huggingface.co/learn, o curso gratuito do Hugging Face

A FRASE (OPÇÃO 7 DO MENU)

Claude Code keeps rewriting my tests without asking.

VADER: SOMA AS PALAVRAS DO LÉXICO

nenhuma palavra do léxico
compound +0.000 -> **neutro**

TRANSFORMER: LÊ A FRASE INTEIRA

neg 78% neu 21% pos 1%
-> **negativo**

0,73
frases sem relação, no estático

0,09
as mesmas, no contextual

Diário de bordo

Quatro erros que apareceram na implementação, e o que cada um exigiu

1

O corte por pontuação emendava listas inteiras

Cortar com o parser do spaCy: 148 frases gigantes viraram 28

2

O ner do spaCy chamava o Claude de pessoa

Treinado com textos de 2013. Uma regra antes do ner marca FERRAMENTA

3

A regra parecia perder 61 issues do Claude

A regex da etapa 1 lia o texto bruto. No mesmo texto limpo: 250 x 244

4

O vizinho mais parecido de "copilot" era um hidroavião

Vetor estático podado: copilot divide o vetor com 29 palavras de aviação

O limite do vetor estático: palavra nova fica de fora

O en_core_web_md guarda 20 mil vetores para 684 mil palavras. A palavra rara usa o vetor de uma frequente.

"chatgpt" e "ollama" nem vetor têm: os vetores vêm de texto coletado em 2021.

Por isso os temas e a busca usam o MiniLM, que lê a frase inteira.

Antes e depois de cada correção

Frases de 300+ palavras

148 → 28

Claude como pessoa

44% → 0%

Diferença regex x regra

61 → 6

Demonstração

python main.py: menu com 10 opções, modelos e resultados em cache, roda sem internet

4

Estático x contextual

O cursor do editor e o do mouse: palavra x sentido

6

Busca semântica

Copyright de código gerado: frases achadas pelo sentido

7

VADER x transformer

A mesma frase lida pelo léxico e pelo RoBERTa

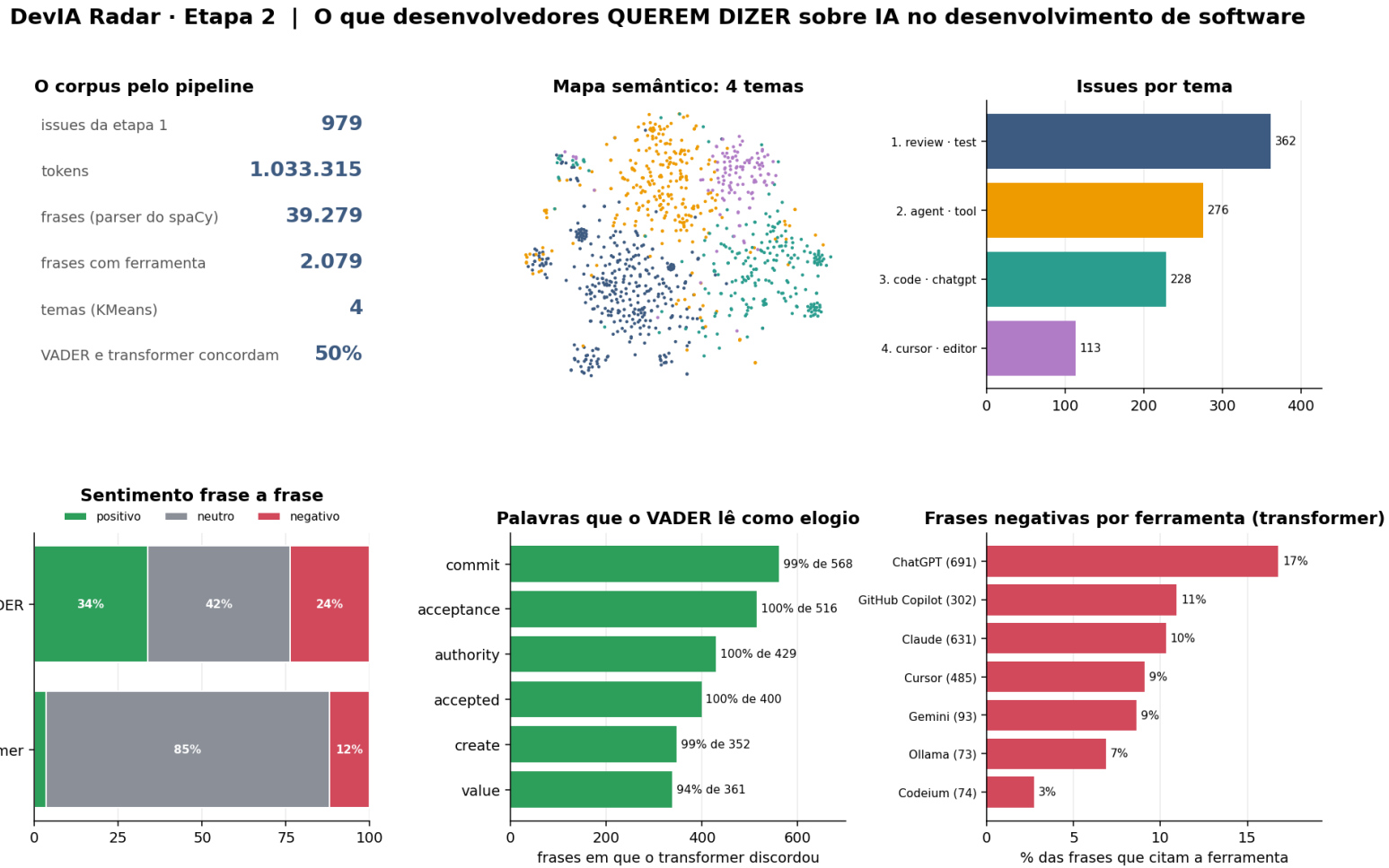
9

Dashboard

Os 9 gráficos, com o painel do próximo slide

```
DevIA Radar - Etapa 2 (semântica) - UFPB
estado: issues:979 | temas:4 | sentimento:ok
----------------------------------------
 [1] Ler o corpus da etapa 1
 [2] Demonstrar o pipeline do spaCy
 [3] Passar o corpus pelo pipeline
 [4] Demonstrar embeddings
 [5] Vetores do corpus e temas
 [6] Busca semântica
 [7] Demonstrar o transformer (x VADER)
 [8] Sentimento de todas as frases
 [9] Gerar gráficos e dashboard
 [0] Rodar o pipeline completo
 [r] Refazer tudo do zero
 [s] Sair
```

Resultados



Painel da opção 9: o corpus, os 4 temas, o tom por frase, as palavras e as ferramentas.

Quase tudo é neutro

85% das frases são neutras para o transformer. Quando há tom, 78% das vezes é queixa.

Jargão vira elogio

commit, acceptance e authority são elogio no VADER. O transformer discordou em 99% dessas frases.

O tema mais negativo

Tema 3, código gerado e licença: 19% de frases negativas. Entre as ferramentas, o ChatGPT: 17%.

Ressalva: VADER e transformer concordam em 50% das frases (kappa 0,16). Nenhum dos dois é gabarito.

Limitações assumidas

Os 4 temas se sobrepõem

Silhueta de 0,08, numa escala de -1 a 1. Parte do tema 1 é o molde de texto de poucos projetos.

O transformer aprendeu com tweets

"Kill the process and restart the server." sai 79% negativa: é uma instrução.

4 repositórios somam 31% das frases

Issue longa pesa mais no total por frase. O tema 1 tem 62% das frases.

As três etapas

1

Texto: 979 issues

O VADER, por issue: 65% positivas e 31% negativas.

2

Sentido: as mesmas issues

O transformer, por frase: 3% positivas e 12% negativas.

3

Áudio e vídeo: 8 palestras

A próxima etapa: a voz e a tela das palestras.

Em que tom? **Neutro em 85% das frases, e a queixa ganha do elogio.**